Lecture 15: BERT Deep Dive

Week 6, Lecture 1 - Bidirectional Encoder Representations

PSYC 51.07: Models of Language and Communication

Winter 2026

Today's Agenda 📋

1. 🎯 **BERT Introduction**: What makes it special?
2. 🎭 **Masked Language Modeling**: The key training objective
3. 🏗️ **BERT Architecture**: Model sizes and specifications
4. 📊 **Pre-training & Fine-tuning**: The two-stage paradigm
5. 🔬 **Contextual Embeddings**: Seeing polysemy in action
6. 💻 **Using BERT**: Practical code examples

Goal: Deep understanding of BERT and how it revolutionized NLP

BERT: Bidirectional Encoder Representations 🎯

BERT = Encoder-only Transformer

Key Innovation: Masked Language Modeling (MLM)

Traditional Language Models:

- Left-to-right (GPT)
- Or right-to-left
- Unidirectional context
- Can't see full picture

Example:

- "The cat sat on the ___"
- Only sees left context

BERT: Bidirectional Encoder Representations 🎯

BERT = Encoder-only Transformer

Key Innovation: Masked Language Modeling (MLM)

Traditional Language Models:

- Left-to-right (GPT)
- Or right-to-left
- Unidirectional context
- Can't see full picture

Example:

- "The cat sat on the ___"
- Only sees left context

BERT (MLM):

- Mask random tokens
- Predict from bidirectional context
- Sees both left & right
- Deeper understanding

Example:

- "The cat [MASK] on the mat"
- Sees: "The cat" AND "on the mat"
- Predicts: "sat"

Reference: Devlin et al. (2019) - "BERT: Pre-training of Deep Bidirectional Transformers"

Why BERT Was Revolutionary 🚀

Before BERT (2018):

- Feature-based approaches (use Word2Vec/GloVe as features)
- Task-specific architectures
- Limited transfer learning
- Unidirectional or shallow bidirectional models

BERT's Contributions:

1. **Deep Bidirectionality**
   - True bidirectional context at every layer
   - Not just concatenating left-to-right and right-to-left
2. **Pre-train + Fine-tune Paradigm**
   - Single pre-trained model for all tasks
   - Fine-tune with minimal architecture changes
   - Democratized NLP (no need to train from scratch!)
3. **State-of-the-Art Results**
   - Beat previous best on 11 NLP tasks
   - Large performance gains (sometimes 10+ points!)
   - Showed power of pre-training

Masked Language Modeling (MLM) 🎭

BERT's Pre-training Objective

Training Procedure:

1. Take a sentence
2. Randomly mask 15% of tokens
3. Of the masked tokens:
   - 80%: Replace with [MASK]
   - 10%: Replace with random word
   - 10%: Keep unchanged
4. Predict the original tokens

Example

Original: "My dog is hairy"
Masked: "My dog is [MASK]"
Labels: [-, -, -, hairy]
Prediction: Model predicts "hairy" using bidirectional context

Why the 80/10/10 split?

- Prevents overfitting to [MASK] token
- Forces model to maintain representations for all tokens

MLM Example: Step by Step 📝

Sentence: "The quick brown fox jumps over the lazy dog"

Step 1: Random Masking (15% of tokens)

The quick brown fox jumps over the lazy dog

Select 2 tokens for masking (15% of 9)

Step 2: Apply Masking Strategy

- "quick" → 80% chance → [MASK]
- "over" → 10% chance → "under" (random word)

Input to BERT: "The [MASK] brown fox jumps under the lazy dog"
Prediction targets: quick, over

Model learns:

- "quick" from context: "The ___ brown" (adjective before noun)
- "over" from context: "jumps ___ the" (preposition in this context)

Next Sentence Prediction (NSP) 🔗

BERT's second pre-training objective (debated usefulness)

Task: Given two sentences A and B, predict if B follows A

Positive Example (IsNext)

Sentence A: "The man went to the store."
Sentence B: "He bought a gallon of milk."
Label: IsNext □

Negative Example (NotNext)

Sentence A: "The man went to the store."
Sentence B: "Penguins are flightless birds."
Label: NotNext □

BERT Architecture Variants 📊

Model	Layers	Hidden Size	Heads	Params
BERT-Base	12	768	12	110M
BERT-Large	24	1024	16	340M

Architecture Details (BERT-Base):

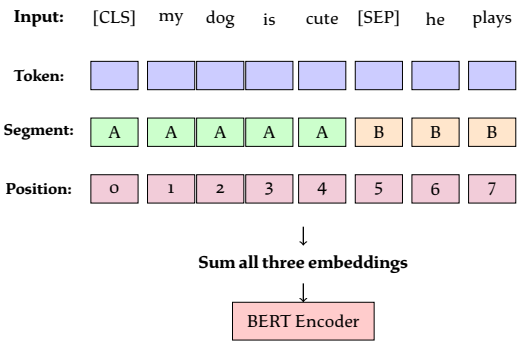
- 12 transformer encoder layers
- 768-dimensional hidden states
- 12 attention heads per layer (64 dims each)
- 3072-dimensional feed-forward intermediate size (4x expansion)
- Maximum sequence length: 512 tokens
- Vocabulary size: 30,000 WordPiece tokens

Special Tokens:

- **[CLS]**: Classification token (first token, used for sequence-level tasks)
- **[SEP]**: Separator token (between sentences)
- **[MASK]**: Mask token (for MLM)
- **[PAD]**: Padding token (for variable-length sequences)

BERT Input Representation

Three types of embeddings are summed:



Three embedding types:

1. **Token Embeddings**: WordPiece vocabulary
2. **Segment Embeddings**: Which sentence (A or B)?
3. **Position Embeddings**: Learned position (0 to 511)

BERT Pre-training 🏋️

Massive scale pre-training on unlabeled text

Pre-training Data:

- **BooksCorpus**: 800M words (novels, fiction)
- **English Wikipedia**: 2,500M words
- Total: 3.3 billion words
- Diverse, high-quality text

Training Details:

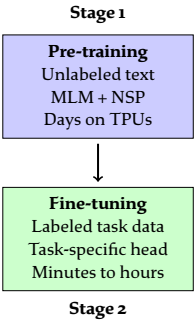
- Batch size: 256 sequences (128,000 tokens)
- Training steps: 1M steps
- Optimization: Adam (lr=1e-4, warmup=10k steps)
- Hardware: 4-16 Cloud TPUs
- Training time: 4 days (BERT-Base), 4+ days (BERT-Large)

Key Insight

Pre-training learns general language understanding that transfers to many downstream tasks!

Fine-tuning BERT 🎓

Two-stage process: Pre-train then Fine-tune



Pre-train once
Use for all tasks

Fine-tune per task:
- Classification
- NER
- Question Answering
- etc.

Benefits:

- Pre-training is expensive but done once
- Fine-tuning is cheap and fast
- Same pre-trained model for all downstream tasks

Fine-tuning for Different Tasks 🎯

Minimal architecture changes needed!

1. **Single Sentence Classification**
   - Input: [CLS] sentence [SEP]
   - Output: [CLS] representation classifier
   - Example: Sentiment analysis
2. **Sentence Pair Classification**
   - Input: [CLS] sentence A [SEP] sentence B [SEP]
   - Output: [CLS] representation classifier
   - Example: Natural Language Inference
3. **Question Answering**
   - Input: [CLS] question [SEP] passage [SEP]
   - Output: Token-level predictions for start/end positions
   - Example: SQuAD
4. **Token Classification**
   - Input: [CLS] sentence [SEP]
   - Output: Each token representation classifier
   - Example: Named Entity Recognition

Fine-tuning BERT: Code Example 💻

Using HuggingFace Transformers

```
from transformers import BertForSequenceClassification, Trainer,
    TrainingArguments

# Load pre-trained BERT with classification head
model = BertForSequenceClassification.from_pretrained(
    'bert-base-uncased',
    num_labels=2  # Binary classification
)

# Define training arguments
training_args = TrainingArguments(
    output_dir='./results',
    num_train_epochs=3,
    per_device_train_batch_size=16,
    learning_rate=2e-5,
    warmup_steps=500,
```

Contextual Embeddings in Action

Remember "bank"? Let's see BERT handle it!

```
from transformers import BertTokenizer, BertModel
import torch

tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
model = BertModel.from_pretrained('bert-base-uncased')

# Two different contexts for "bank"
sent1 = "I deposited money at the bank"
sent2 = "We sat by the river bank"

# Get embeddings
def get_embedding(sentence, target_word):
    inputs = tokenizer(sentence, return_tensors='pt')
    outputs = model(**inputs)
    # Find position of target word
    tokens = tokenizer.tokenize(sentence)
```

BERT's Impressive Results 

State-of-the-art on 11 NLP tasks when released (2018)

Task	Metric	Pre-BERT	BERT
SQuAD 1.1 (QA)	F1	84.1	**90.9**
SQuAD 2.0 (QA)	F1	66.3	**83.1**
MNLI (NLI)	Accuracy	80.6	**86.7**
SST-2 (Sentiment)	Accuracy	93.2	**94.9**
CoNLL-2003 (NER)	F1	92.6	**92.8**

Key Observations:

- Largest gains on tasks requiring understanding (QA, NLI)
- Improvements even on well-studied benchmarks
- BERT-Large generally better than BERT-Base
- Fine-tuning is simple but very effective

What Does BERT Learn? 🧠

Probing BERT's internal representations

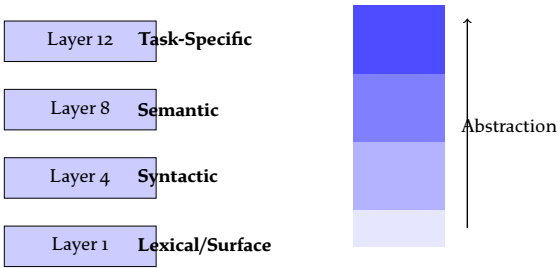
Research has shown BERT captures:

1. **Syntactic Information**
   - Part-of-speech tags
   - Constituent structure
   - Dependency relations
   - Lower layers encode more syntax
2. **Semantic Information**
   - Word sense disambiguation
   - Semantic roles
   - Entity types
   - Middle layers encode more semantics
3. **Pragmatic Information**
   - Coreference resolution
   - Discourse relations
   - Higher layers encode more pragmatics
4. **World Knowledge**
   - Factual knowledge (to some extent)
   - Common sense reasoning (limited)

Reference: Tenney et al. (2019) - "BERT Rediscovers the Classical NLP Pipeline"

BERT Layer Analysis 📊

Different layers capture different linguistic properties



Observations:

- Lower layers: surface features (word forms, POS)
- Middle layers: syntax (phrase structure, dependencies)
- Higher layers: semantics and task-specific features
- Hierarchical representation learning

Similar to how CNNs learn in computer vision: edges → shapes → objects

Discussion Questions 💭

1. **MLM vs. Autoregressive:**
   - Why is MLM better for understanding tasks?
   - Can BERT generate text like GPT?
   - What are the trade-offs?
2. **The 80/10/10 Masking Strategy:**
   - Why not just use 100% [MASK]?
   - What problem does the random replacement solve?
   - Could we improve this strategy?
3. **Pre-training Data:**
   - Why use books and Wikipedia?
   - Would social media text work as well?
   - How does data quality affect pre-training?
4. **Fine-tuning:**
   - Why does fine-tuning work so well?
   - When might fine-tuning fail?
   - How much labeled data do we need?

Looking Ahead 🔮

Today we learned:

- BERT architecture and innovations
- Masked Language Modeling
- Pre-training and fine-tuning
- Contextual embeddings
- What BERT learns

Next lecture (Lecture 16 - BERT Variants):

- RoBERTa: Optimized BERT training
- ALBERT: Parameter-efficient BERT
- DistilBERT: Smaller, faster BERT
- Other variants: ELECTRA, DeBERTa, and more
- Comparative analysis and when to use which

BERT started a revolution in NLP! 🚀

Summary 🎯

Key Takeaways:

1. **BERT = Encoder-only Transformer**
   - Bidirectional self-attention
   - Trained with Masked Language Modeling
2. **Pre-train + Fine-tune Paradigm**
   - Expensive pre-training on unlabeled data (once)
   - Cheap fine-tuning on task-specific data (per task)
3. **Contextual Embeddings**
   - Different representations based on context
   - Solves polysemy problem
4. **Hierarchical Learning**
   - Lower layers: syntax
   - Higher layers: semantics
   - Learns linguistic structure automatically
5. **Revolutionary Impact**
   - Established pre-training as standard
   - Democratized NLP research
   - Foundation for modern LLMs

References

Essential Papers:

- **Devlin et al. (2019)** - "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding"
  - The original BERT paper
  - Introduced MLM and NSP
- **Tenney et al. (2019)** - "BERT Rediscovers the Classical NLP Pipeline"
  - Analysis of what BERT learns
  - Layer-wise linguistic properties
- **Clark et al. (2019)** - "What Does BERT Look At? An Analysis of BERT's Attention"
  - Understanding BERT's attention patterns

Tutorials:

- HuggingFace Course: Chapter 1 (Transformer Models)
- Jay Alammar: "The Illustrated BERT, ELMo, and co."
- BertViz: Interactive attention visualization

Questions? 🙋

Discussion Time

Topics for discussion:

- Masked Language Modeling
- Pre-training vs fine-tuning
- Contextual embeddings
- BERT architecture details
- Implementation questions

Thank you! 🙏

Next: BERT Variants and Improvements!